Introduction to Neural Networks

Lecture Notes — CS 4820: Foundations of Artificial Intelligence

1. What is a Neural Network

A neural network is a computational model loosely inspired by the structure of biological brains. It consists of layers of interconnected nodes, or neurons, each of which receives input, applies a mathematical transformation, and passes its output to the next layer. The first layer is called the input layer, the last is the output layer, and any layers in between are called hidden layers. The depth of a network refers to the number of these layers, and networks with many hidden layers are commonly referred to as deep neural networks.

Each connection between neurons carries a weight, a numerical value that determines how strongly one neuron influences another. During training, these weights are adjusted so that the network produces increasingly accurate outputs for a given set of inputs. The network also uses bias terms, which shift the activation of a neuron independently of its inputs. Together, weights and biases define the parameters of the model, and learning is the process of finding parameter values that minimise prediction error.

Neural networks are particularly powerful because they are universal function approximators. Given enough neurons and layers, a neural network can theoretically approximate any continuous function to arbitrary precision. This property makes them applicable to a wide range of problems including image classification, natural language processing, speech recognition, and game playing. However, realising this potential in practice requires careful design choices around architecture, training data, and optimisation strategy.

2. Activation Functions

Activation functions introduce non-linearity into a neural network. Without them, stacking multiple linear layers would be mathematically equivalent to a single linear transformation, severely limiting the kinds of functions the network could represent. An activation function is applied to the output of each neuron before passing it along, allowing the network to learn complex, curved decision boundaries rather than only straight hyperplanes.

The sigmoid function was historically popular because it maps any real number to a value between zero and one, making it interpretable as a probability. However, it suffers from the vanishing gradient problem: for very large or very small inputs, the gradient of the sigmoid becomes extremely small, causing updates to the weights in earlier layers to be negligible.

This made training deep networks with sigmoid activations slow and unreliable.

The Rectified Linear Unit, or ReLU, has largely replaced sigmoid in hidden layers. ReLU outputs the input directly if it is positive, and zero otherwise. This simple function alleviates the vanishing gradient problem for positive activations and is computationally cheap to evaluate. Variants such as Leaky ReLU and GELU have been developed to address the issue of dead neurons, which occur when a ReLU unit outputs zero for all inputs and therefore stops contributing to learning entirely.

3. Backpropagation

Backpropagation is the algorithm used to train neural networks by computing the gradient of the loss function with respect to each weight in the network. The process begins with a forward pass, in which input data is fed through the network to produce a prediction. The prediction is then compared to the true label using a loss function, such as cross-entropy for classification or mean squared error for regression, producing a scalar measure of how wrong the prediction was.

The backward pass then applies the chain rule of calculus to propagate the error signal from the output layer back through each hidden layer to the input. At each layer, the gradient tells us how much each weight contributed to the overall error and in which direction it should be adjusted to reduce that error. This information is used by an optimiser, most commonly stochastic gradient descent or Adam, to update the weights by a small step proportional to the negative gradient.

The efficiency of backpropagation comes from its use of dynamic programming: intermediate gradient values computed during the backward pass are reused rather than recomputed, making the algorithm run in time linear in the number of parameters. Modern deep learning frameworks such as PyTorch and TensorFlow implement automatic differentiation, which means they can compute gradients for arbitrary computational graphs without the programmer needing to derive the gradients manually.

4. Overfitting and Regularization

Overfitting occurs when a model learns the training data too well, capturing noise and incidental patterns that do not generalise to unseen examples. An overfit model will achieve very low training loss but perform poorly on the validation or test set. Neural networks are especially prone to overfitting because their large number of parameters gives them the capacity to memorise training examples rather than learning the underlying distribution.

Regularisation techniques are methods that constrain or penalise model complexity to improve generalisation. L2 regularisation, also known as weight decay, adds a penalty proportional to the squared magnitude of the weights to the loss function, encouraging the model to keep weights small. Dropout is a technique where, during training, random neurons are temporarily set to zero with a given probability, forcing the network to learn redundant representations and preventing any single neuron from becoming overly influential.

Early stopping is another practical regularisation strategy: training is halted when performance on the validation set stops improving, even if the training loss is still decreasing. Data augmentation artificially expands the training set by applying random transformations such as rotations, flips, or colour jitter to existing examples, making the model more robust to variation. Batch normalisation, while primarily designed to accelerate training, also acts as a mild regulariser by adding noise to activations during training.

5. Convolutional Neural Networks

Convolutional Neural Networks, or CNNs, are a specialised architecture designed for processing data with a grid-like topology, most commonly images. Rather than connecting every neuron to every input as in a fully connected layer, a convolutional layer applies a small learned filter, called a kernel, across the entire input by sliding it over spatial positions and computing a dot product at each location. This operation is called convolution and produces a feature map that highlights where in the image a particular pattern was detected.

Two key properties make convolutional layers well-suited to image data. The first is parameter sharing: the same kernel weights are used at every spatial position, drastically reducing the number of parameters compared to a fully connected layer and making the network more efficient. The second is translation equivariance: if an object shifts position in the input, its representation in the feature map shifts correspondingly, meaning the network does not need to relearn a feature for every possible location it might appear.

A typical CNN architecture alternates convolutional layers with pooling layers, which downsample the feature maps by taking the maximum or average value within small windows. This reduces spatial resolution progressively while increasing the depth of feature representations. After several such stages, the feature maps are flattened and passed through one or more fully connected layers to produce the final output. Landmark CNN architectures such as AlexNet, VGGNet, ResNet, and EfficientNet have driven major advances in computer vision over the past decade.